

Day 16

Methods

- It is a block of code that performs a specific task.
- A method runs or executes only when it is called.
- Methods provide for easy modification and code reusability. It will get executed only when invoked/called.

Method Signature / Method Prototype / Method Definition

A method in Java has various attributes like **access modifier**, **return type**, **name**, **parameters** etc.

Methods can be declared using the following syntax:



```
accessModifier returnType methodName(parameters..){  
    //logic of the function  
}
```

Example :



```
public static int sum(int a, int b) {  
    int sum = a+b;  
    return sum;  
}
```

Here,

public - access modifier

static - special access modifier

int - return type

sum - method name

int a and int b - parameter

- **accessModifier:** It defines the method's access type, i.e., from where it can be accessed in your application.
- **returnType:** It represents the data type of the value returned by the function. For example, a method in Java declared with int return type should return an integer value.
- **methodName:** Represents the identifier that can be used to call the method when required.
- **parameters:** These are the arguments passed into a method necessary for the function's logic. We can pass data to the methods by specifying them within the parentheses if the methods have data.

Methods mainly are of two type -

- Static
- Non - Static

Static Method

- A method declared as static does not need an object of the class to invoke it.
- All the built-in methods are static - min, max, sqrt etc. called using Math class name

Example :

```
public class Demo {  
    public static void main(String args[]){  
        Demo.sum(1,2);//call by classname  
    }  
    // static method  
    public static int sum(int a, int b){  
        int sum = a+b;  
        return sum;  
    }  
}
```

Non-Static Method or Instance Method

- Non-Static Method or Instance methods are attached to the objects of a class, rather than the class itself.
- In simple words, you need to create an object to invoke them.

Example :

```
public class Demo {  
    public void main(String args[]){  
        Demo obj = new Demo();  
        obj.sum(1,2);//call by object reference  
    }  
    // static method  
    public static int sum(int a, int b){  
        int sum = a+b;  
        return sum;  
    }  
}
```

Day 15

Array

An array is a linear data structure used to store a collection of elements of the same data type in contiguous memory locations.

Arrays in Java are **non-primitive** data types and it can store both primitive and non-primitive types of data in it. They are fixed in size, meaning that when you create an array you need to give specific size and you cannot change the size later.

Declaration of Array



```
DataType[] arrayName;
```

Creating an Array

After declaring an array, you need to create an actual instance of the array with a specific size using the new keyword. For example, to create an array of integers with a size of 5:



```
int[] numbers = new int[5];
```

Size and initialization can't be done together

```
int[] arr = new int[3]{1, 2, 3}; // compilation err
```



```
// You can do this  
int[] arr = new int[]{1, 2, 3};  
int[] arr = {1, 2, 3};
```

Stack memory holds the references while Heap memory holds the actual object:

Stack: It is memory in which the size of the stack is limited and predefined during program execution. Exceeding this limit can result in a **StackOverflowError(DTL)**.

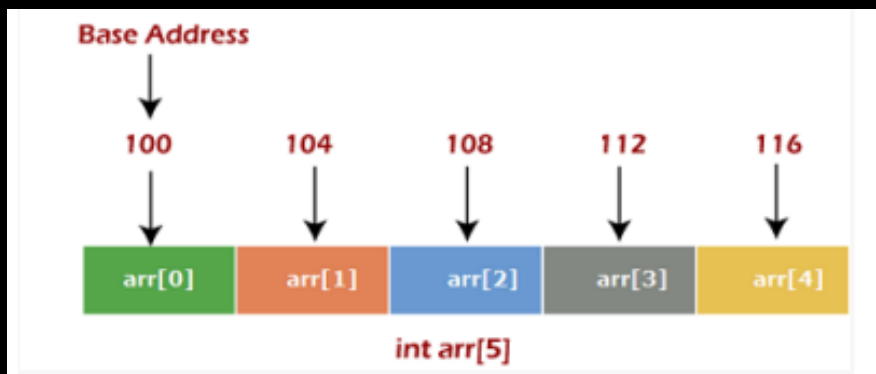
Heap: The heap memory in Java can grow and shrink dynamically(DTL).

Reference of the array is stored on the stack(int[] arr).

Reference is essentially a memory address that points to the location in the heap where the actual array object is stored.

Address

Contiguous Elements: The elements of the array are stored in contiguous memory locations. This means that the memory addresses for each element are sequential. The memory address of the first element in the array is the base address of the array.



Internally the address is in hexadecimal number (combination of alphabets & numeric characters) this is just for your understanding (we take 100,104,108 etc).

In Java you cannot access the memory directly, you generally work with higher-level abstractions, and the specifics of memory addresses are hidden from you & handled by the **Java Virtual Machine (JVM)**.

Elements in the array are accessed by their index. When you use an index to access an element, Java calculates the memory address of that element using the base address of the array and the size of the elements.

Address = Base address + (index * 4)

Address of 1st index = $100 + (1 \times 4) = 104$

Enhanced for loop || for-each loop

The for-each also called as enhanced for loop, was introduced in Java 5. It is one of the alternative approaches that is used for traversing arrays. Traverse the array without using the index & makes the code simple as it reduces the code length.

Syntax:

```
for(datatype element : arrayName) {  
    // Code  
}
```

Example:

```
int arr[] = {1, 2, 3};  
for (int elem : arr) {  
    System.out.print(elem + ", ");  
}
```

Day 17

Arguments

An argument is a value passed to a function when the function is called.

A parameter is a variable used to define a particular value during a method definition.

In common we call both parameter and argument as either parameter/argument.

Classification of Arguments

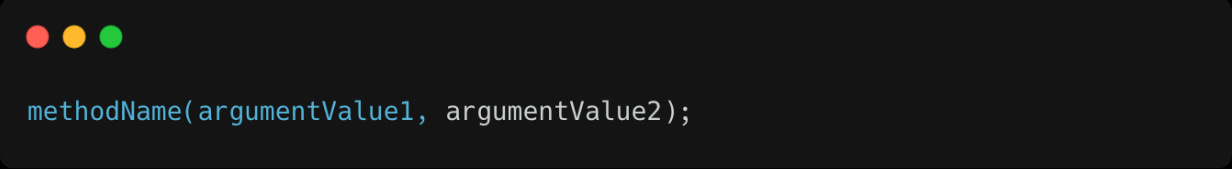
Formal argument : The identifier used in a method at the time of method definition.

Example :

```
returnType methodName(dataType parameterName1, dataType p2) {  
    // body  
}
```

Actual argument : The actual value that is passed into the method at the time of method calling.

Example :



```
methodName(argumentValue1, argumentValue2);
```

Arguments Passing

1. Pass By Value:

When we pass only the value part of a variable to a function as an argument, it is referred to as pass by value.

Any change to the value of a parameter in the called method does not affect its value in the calling method.

As can be seen in the figure below, only the value part of the variable is passed i.e. a copy of the existing variable is passed instead of passing the origin variable. Hence, any changes done to the value of the copy will not have any impact on the value of the original variable. Java supports pass-by-value.

Example :

```
public class Main{
    public static void solve(int a){
        a = a+10; // changes inside called method
    }
    public static void main(String[] args) {
        int a = 10;
        System.out.println("Value of a "+ a);
        solve(a);
        System.out.println("Value of a "+ a);
    }
}
```

Output : Value of a 10
Value of a 10

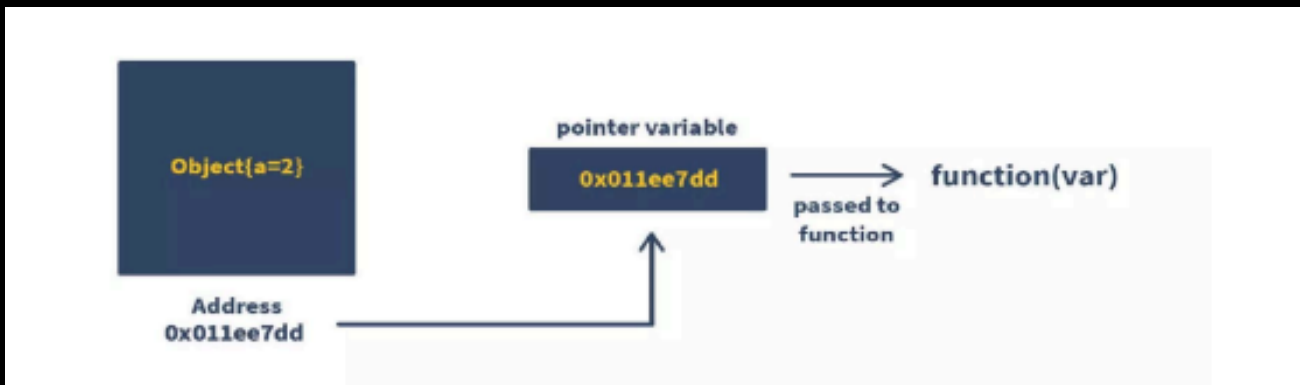
2. Pass by reference: Not supported by Java

In pass-by-reference, changes made to the parameters inside the method are also reflected outside. Though **Java does not support pass-by-reference**.

In Java, when we create a variable of class type or non primitive , the variable holds the reference to the object in the heap memory. This reference is stored in the stack memory. The method parameter that receives the object refers to the same object as that referred to by the argument.

Thus, changes to the properties of the object inside the method are reflected outside as well. This effectively means that objects are passed to methods by use of **call-by-reference**.

Changes to the properties of an object inside a method affect the original argument as well. However, if we change the object altogether, then the original object is not changed. Instead a new object is created in the heap memory and that object is assigned to the copied reference variable passed as argument.



Pass by Value for non-primitives

Example :

```
class PassByValue {
    public static void main(String[] args) {
        int[] array = new int[2];
        array[0] = 2;
        array[1] = 3;
        add(array);
        System.out.println("Result from main: " + (array[0] + array[1]));
    }
    private static void add(int[] array) {
        array[0] = 10;
        System.out.println("Result from method: " + (array[0] + array[1]));
    }
}
```

Output : Result from method: 13

Result from main: 13

The called method is able to modify the original object but not replace it with another object.

Example :

```
class PassByValue {  
    public static void main(String[] args) {  
        Integer[] array = new Integer[2];  
        array[0] = 2;  
        array[1] = 3;  
        add(array);  
        System.out.println("Result from main: " + (array[0] + array[1]));  
    }  
    public static void add(Integer[] array) {  
        array = new Integer[2];  
        array[0] = 10;  
        array[1] = 3;  
        System.out.println("Result from method: " + (array[0] + array[1]));  
    }  
}
```

Output : Result from method: 13

Result from main: 5

Here, we can see that if we reinitialize the array object, which is passed in arguments, the original reference breaks(i.e., we have replaced the original object reference with some other object reference), and the array no longer is referenced to the original array. Hence the value in the main() method didn't change.

Points to Remember

- Java supports pass-by-value only.
- **Java doesn't support pass-by-reference.**
- Primitive data types and Immutable class objects strictly follow pass-by-value; hence can be safely passed to functions without any risk of modification.
- For non-primitive data types, Java sends a copy of the reference to the objects created in the heap memory.
- Any modification made to the referenced object inside a method will reflect changes in the original object.
- If the referenced object is replaced by any other object, any modification made further will not impact the original object.

Varargs (...)

Varargs also known as **variable arguments** is a method that takes input as a variable number of arguments.

The varargs method is implemented using a single dimension array internally. Hence, arguments can be differentiated using an index. A variable-length argument can be specified by using three-dot (...) or periods.

Syntax



```
public static void solve(int ... a){  
    // method body  
}
```

Rules

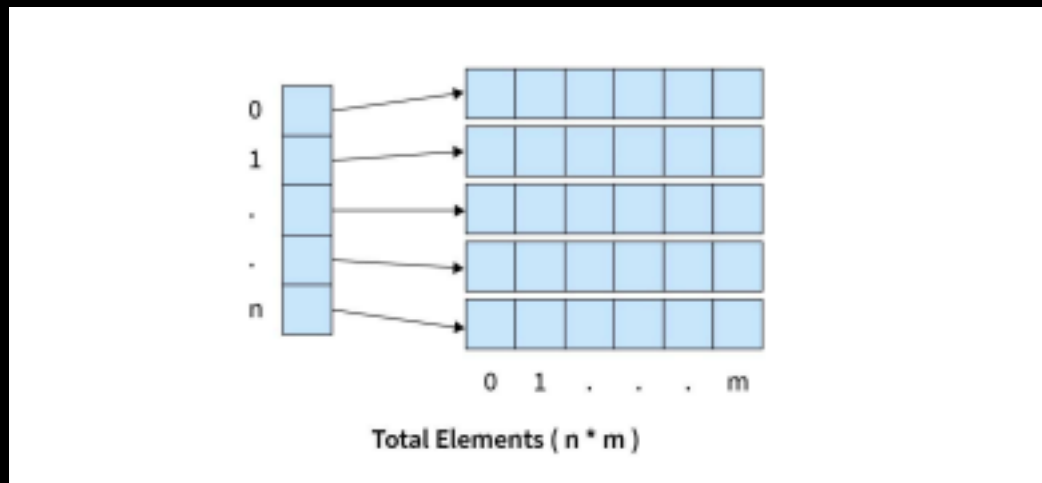
- There can be only one varargs in a method.
- If there are other parameters then varargs must be declared in the last.

Day 18

Multi D Arrays

Multidimensional Arrays can be thought of as an array inside the array i.e. elements inside a multidimensional array are arrays themselves.

Multidimensional arrays, like a 2D array, are a bunch of 1D arrays put together in an array. A 3D array is like a bunch of 2D arrays put together in a 1D array, and so on



To access array elements in multidimensional arrays, more than one index is used.

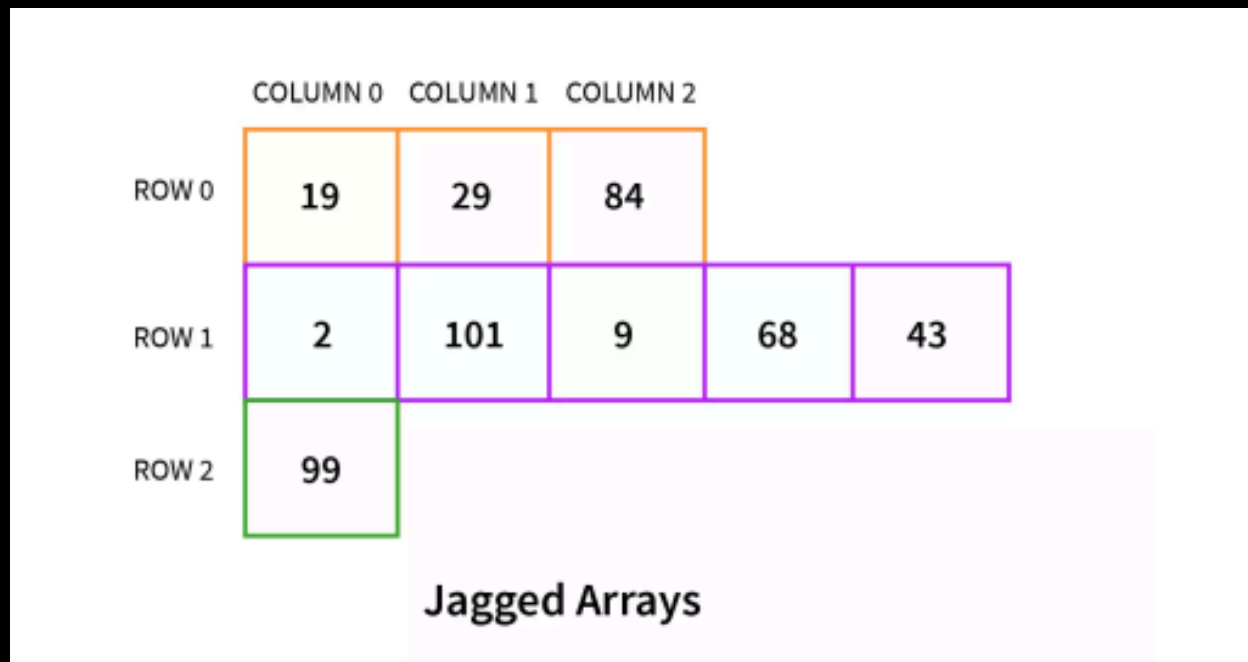
General syntax to initialize the array:

```
arrayName = new DataTye[length 1][length 2]...[length N];
```

```
int[ ][ ] twodArray= new int[3][3];
```

Jagged Arrays

A jagged array is an array of arrays where the inner arrays can have different lengths. Each row of a jagged array can store arrays of varying sizes, making it a "non-rectangular" array.



Example :

```
int[][] jaggedArray = new int[3][]; // Declaring a jagged array
with 3 rows
jaggedArray[0] = new int[4]; // First row has 3 elements
jaggedArray[1] = new int[5]; // Second row has 5 elements
jaggedArray[2] = new int[1]; // Third row has 1 elements
```

Accessing Elements: Access elements using two indices, just like in 2D arrays.

int value = jaggedArray[1][2]; // Accessing the 3rd element of the 2nd row

Day 19

OOPS Introduction

OOPs is a programming paradigm (procedure or method)to solve real world problems.

It is a way of organizing and designing code to model real-world entities and their interactions. The main purpose of OOPs programming is to implement ideas and solve real-world problems.

OOPs (Object Oriented Programming System) refer to languages that use objects in programming.

Main pillars of OOPs:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Classes and objects are the building blocks of an object-oriented programming language.

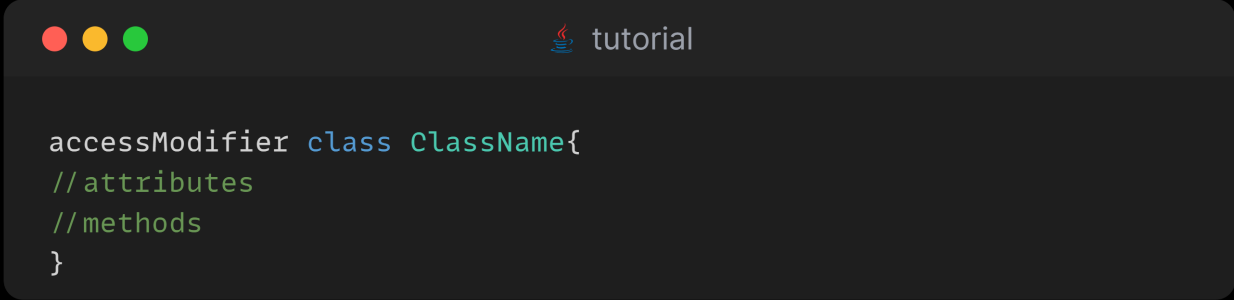
Class:

- If we want to store different types of data we use class.
- Class is a user-defined or customized data type.

- Class is not a real world entity. It is just a template or blueprint or prototype of an object.
- Also, it doesn't occupy any memory.
- Class is a collection of objects.

For example – Animal, car, Birds etc. all are categories not a real world entity.

Syntax:



```
accessModifier class ClassName{  
    //attributes  
    //methods  
}
```

Allowed Access Modifier in class

- Classes can have four different access modifiers: **public, default, abstract and final (DTL).**
- you can only use one of these modifiers at a time to define the accessibility and behavior of your classes.
- only one public class is allowed per file in Java, multiple non-public classes can coexist within the same file.

Object:

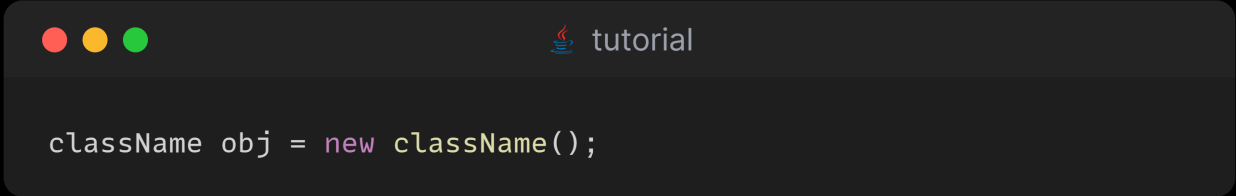
- Object is an instance(part) of a class.
- Object is a real world entity.
- Object occupies memory.

For example – Dog, cat (type of animal). Verna, MG hector, Jeep (type of car).

Object consist of-

- Identity - Unique Name
- State | Attribute - color, breed, age [DOG] (represent by variable)
- Behavior – run, eat, bark etc [DOG] (represent by methods)

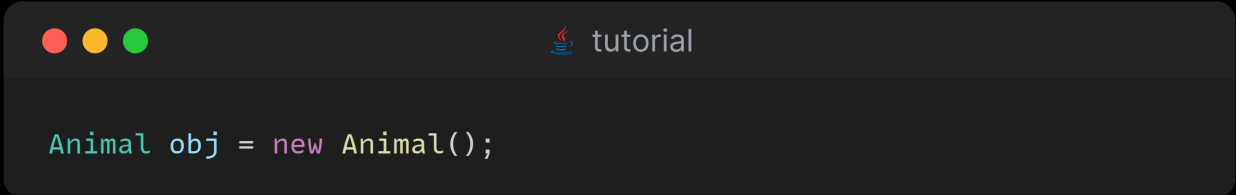
Syntax:



```
className obj = new className();
```

The new keyword is responsible for allocating memory for objects.

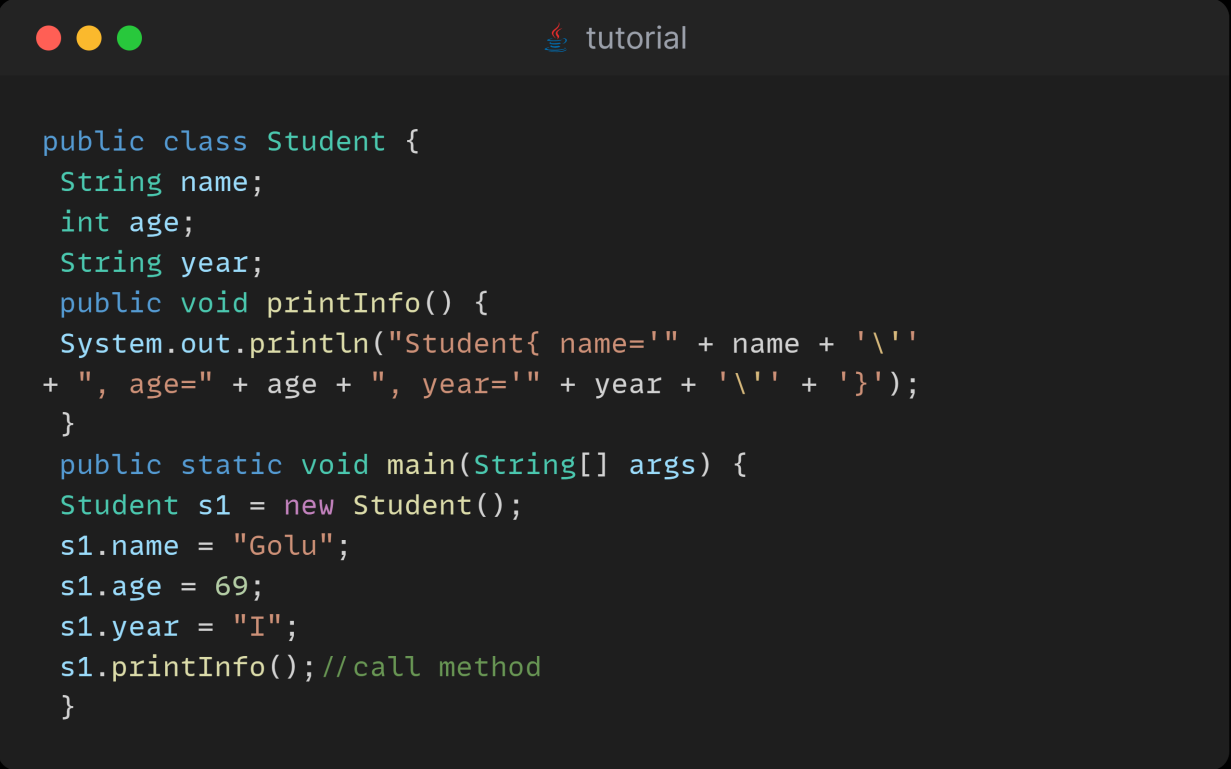
Example:



```
Animal obj = new Animal();
```

Here, Animal () is a **constructor**.

Lets, understand this through the example



```
public class Student {
    String name;
    int age;
    String year;
    public void printInfo() {
        System.out.println("Student{ name='" + name + '\''
+ ", age=" + age + ", year='" + year + '\'' + '}');
    }
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Golu";
        s1.age = 69;
        s1.year = "I";
        s1.printInfo();//call method
    }
}
```

Here, we initialize object by reference

If we want to initialize multiple variables then initializing by reference is not an efficient way.

Then we use a constructor

Constructor:

- It is a special type of method.
- Called at the time of object creation.
- Responsible for initializing the object.
- It can be used to initialize the data member/ attribute of objects.

Rules-

- Same name as the class
- Never have any return type not even void.
- Cannot make static.
- Can have access modifiers, which control the visibility of the constructor.

Allowed access modifier in access modifier

- public, default, , private (when you don't want instances to be created outside the class), protected.
- Cannot be made final (can't override constructor makes no sense), abstract (must have an implementation and cannot be abstract).

Types-

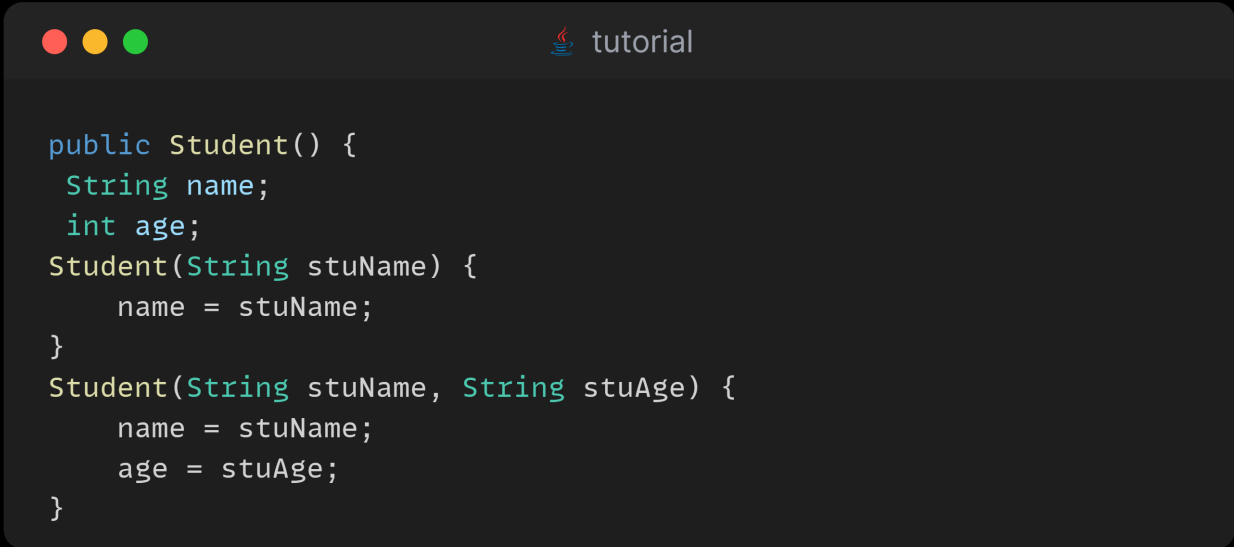
1 . Default | No-Arg Constructors | No Parameterized Constructor

- Do not have any arguments.
- Created by default in Java when no constructors are written by the programmer.

2. Parameterized Constructor

- Constructors with one or more arguments..
- It is possible to write multiple constructors for a single class.

For example



```
public Student() {  
    String name;  
    int age;  
    Student(String stuName) {  
        name = stuName;  
    }  
    Student(String stuName, String stuAge) {  
        name = stuName;  
        age = stuAge;  
    }  
}
```

When you use the same name for data members (instance variables) and constructor parameters in a class, it can lead to ambiguity for the compiler.

In such cases, you can use the "this" keyword to clarify which variable you are referring to.



 tutorial

```
public Student() {  
    String name;  
    int age;  
    Student(String stuName) {  
        name = stuName;  
    }  
    Student(String stuName, String stuAge) {  
        name = stuName;  
        age = stuAge;  
    }  
}
```

this keyword helps the compiler understand whether you are working with the local parameter or the instance variable that shares the same name.

- Represent the current calling object.

Day 21

String API

- If we need to store any name or a sequence of characters then we use String.
- String is an array of characters or sequence of characters.
- Java platform provides the String class to create strings.

Syntax

```
String stdName = "Pappu";
```

||

||

||

DataType

variableName

Array of Characters

String str = "abc"; is equivalent to: char data[] = {'a', 'b', 'c'};

String is an array of characters. Let us see how to create string objects. String object can be created using two ways:

1. Using String Literal.
2. Using new keywords.

Using String Literal and String Constant Pool

A **literal**, in computer science, is a notation used for representing a value. String literal can be created and represented using the double-quotes. All of the content/characters can be added in between the double quotes.

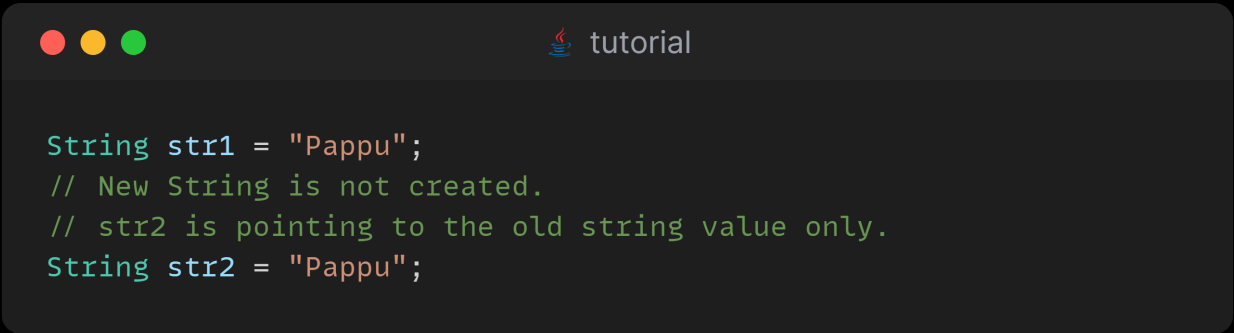
For example

String name = "Golu";

Strings are stored in a special place in the heap called "String Constant Pool" or "String Pool".

String Constant Pool

- The string constant pool is a storage area in the heap memory that stores string literals.
- When a string is created, the JVM checks if the same value exists in the string pool.
- If it does, the reference to that existing object is returned. Otherwise, a new string object is created and added to the string pool, and its reference is returned.



```
String str1 = "Pappu";  
// New String is not created.  
// str2 is pointing to the old string value only.  
String str2 = "Pappu";
```

Using New Keyword

Strings can be created using the new keyword. When a string is created with new, a new object of the String class is created in the heap memory, outside the string constant pool.

Unlike string literals, these objects are allocated separate memory space in the heap, regardless of whether the same value already exists in the heap or not.

Syntax : String str = new String("string_value");

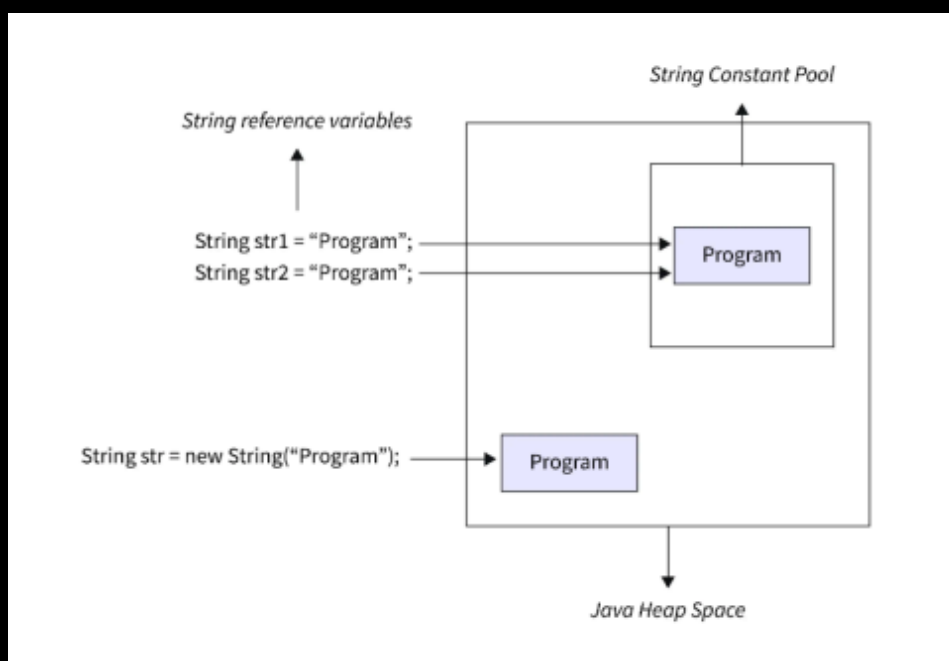
Example:

```
String str1 = new String("Pappu");  
// New String is created.  
// str2 is pointing to the new string value.  
String str2 = new String("Pappu");
```

Let's understand through example

```
String str1 = "Program";  
String str2 = "Program";  
String s = new String("Program");
```

In memory it stored like this -



Methods of Java Strings

length(),
charAt(int index),
contains(),
toUpperCase(),
toLowerCase(),
equals(),
split() etc.
substring(int beginIndex, int endIndex[optional]) -

Example :

s = "shery"
s.substring(0, 5) -> shery
s.substring(0) -> shery (behind the seen it works like
s.substring(0,s.length())
s.substring(5) -> empty string [substring(5 , 5)]

For more methods : [VISIT](#)

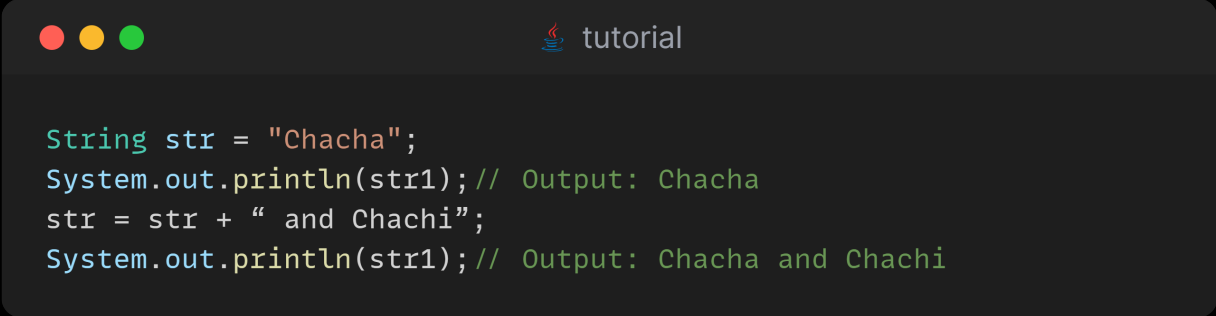
Comparing Strings

- Avoid using == (compare value and address both)
- Use **equals()** (it compares only value)
- **compareTo()** :[string1.compareTo(string2)] It returns a +ve integer if string1 is greater than string2, -ve if string2 is greater than string1, and zero if both are equal.

Java Strings: Mutable or Immutable

- Strings are immutable.
- Means their values cannot be changed once initialized.

Example



```
String str = "Chacha";  
System.out.println(str1); // Output: Chacha  
str = str + " and Chachi";  
System.out.println(str1); // Output: Chacha and Chachi
```

Above, when we concatenate a string " and Chachi" with str, a new string value is created. str then points to this newly created value, while the original value remains unchanged or the actual string value remains unchanged. This behavior demonstrates the immutability of strings.

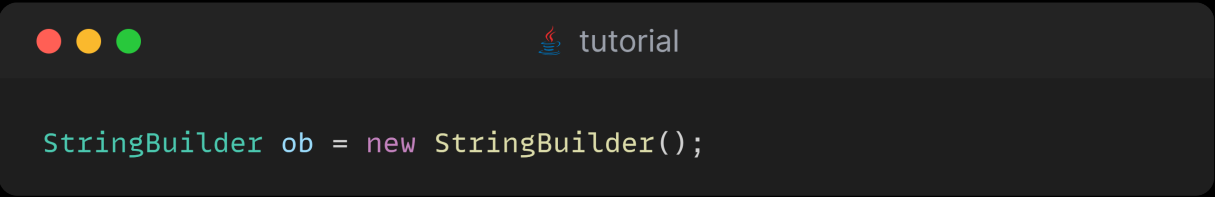
Day 22

StringBuilder

StringBuilder in Java is an alternative to the String class.

It is used for storing the **mutable** (changeable) sequence which means we can update the elements of the StringBuilder class without creating a new StringBuilder sequence in memory.

Syntax



```
StringBuilder ob = new StringBuilder();
```

Default Capacity

Method - `StringBuilder.Capacity()`

- **16 bytes** is the default capacity of the StringBuilder when StringBuilder contains no elements.
- When the StringBuilder capacity gets full. Internally StringBuilder updates the capacity by $(\text{previous Capacity} + 1) * 2$.
- `StringBuilder.length()` and `StringBuilder.capacity()` are the two different methods.

Constructors of StringBuilder

<code>StringBuilder()</code>	-	Generates Empty String Builder with 16-character capacity.
<code>StringBuilder(int capacity)</code>	-	Empty String Builder with the provided length capacity.
<code>StringBuilder(String)</code>	-	The provided string is used to generate a String Builder. Length of string + 16 characters capacity.
<code>StringBuilder(char)</code>	-	Generates Empty String Builder with capacity char ASCII

Methods of StringBuilder

<code>append(String s)</code>	-	append the specified string to the provided string.
<code>insert(int offset, String s)</code>	-	insert the provided string at the designated place.
<code>replace(int startIndex,int endIndex,String str)</code>	-	string from the startIndex and endIndex values are replaced using this method.
<code>delete(int startIndex, int endIndex)</code>	-	delete the string from startIndex and endIndex that are provided.
<code>reverse()</code>	-	It is used to reverse the string.
<code>capacity()</code>	-	return the current capacity.
<code>ensureCapacity(int cap)</code>	-	make sure that the capacity is at least equal to the cap(input).
<code>substring(int beginIndex)</code>	-	substring starting at the beginIndex till end is returned using it.
<code>substring(int beginIndex, int endIndex)</code>	-	retrieve the substring starting at the beginIndex and endIndex values.